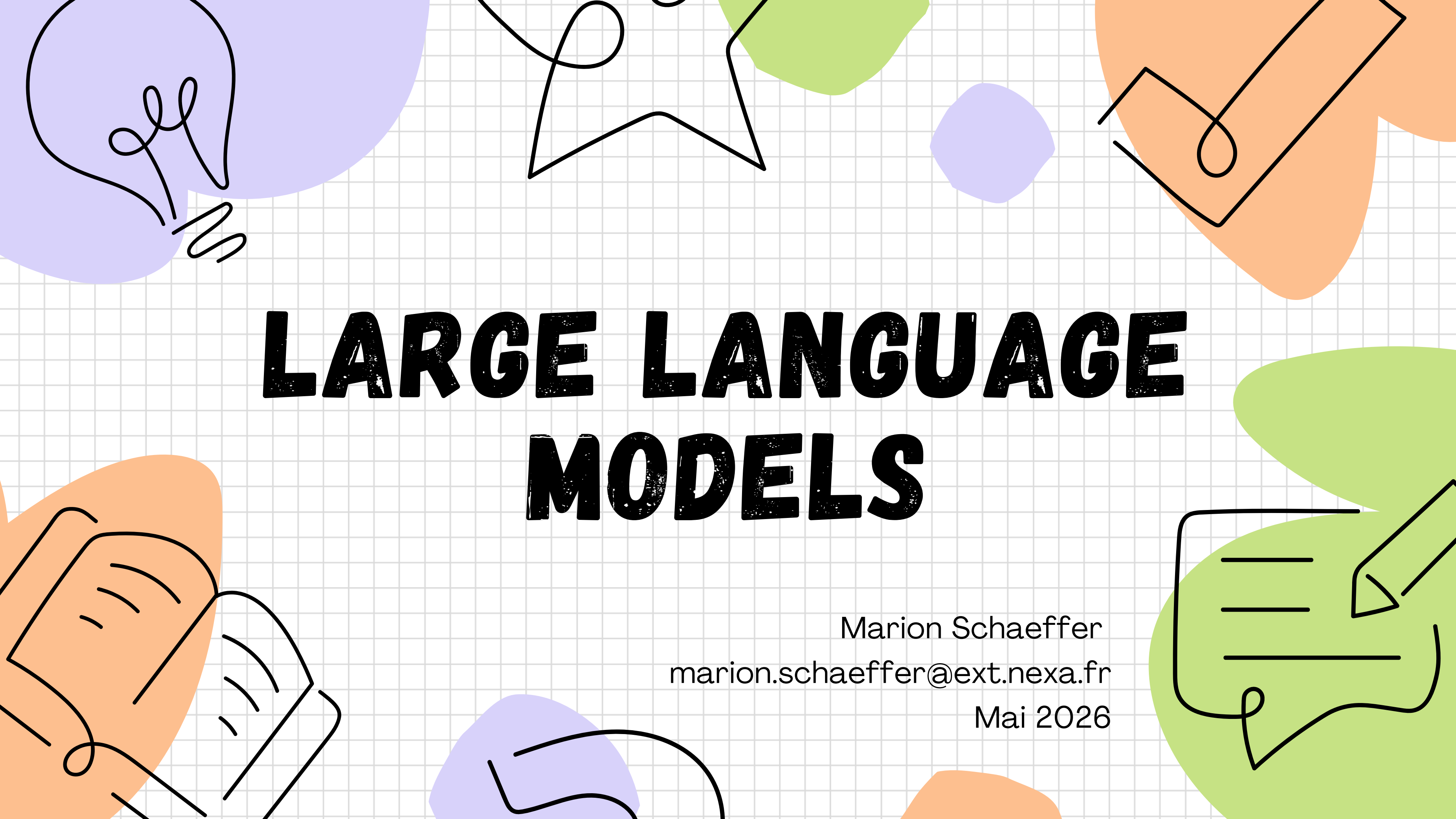
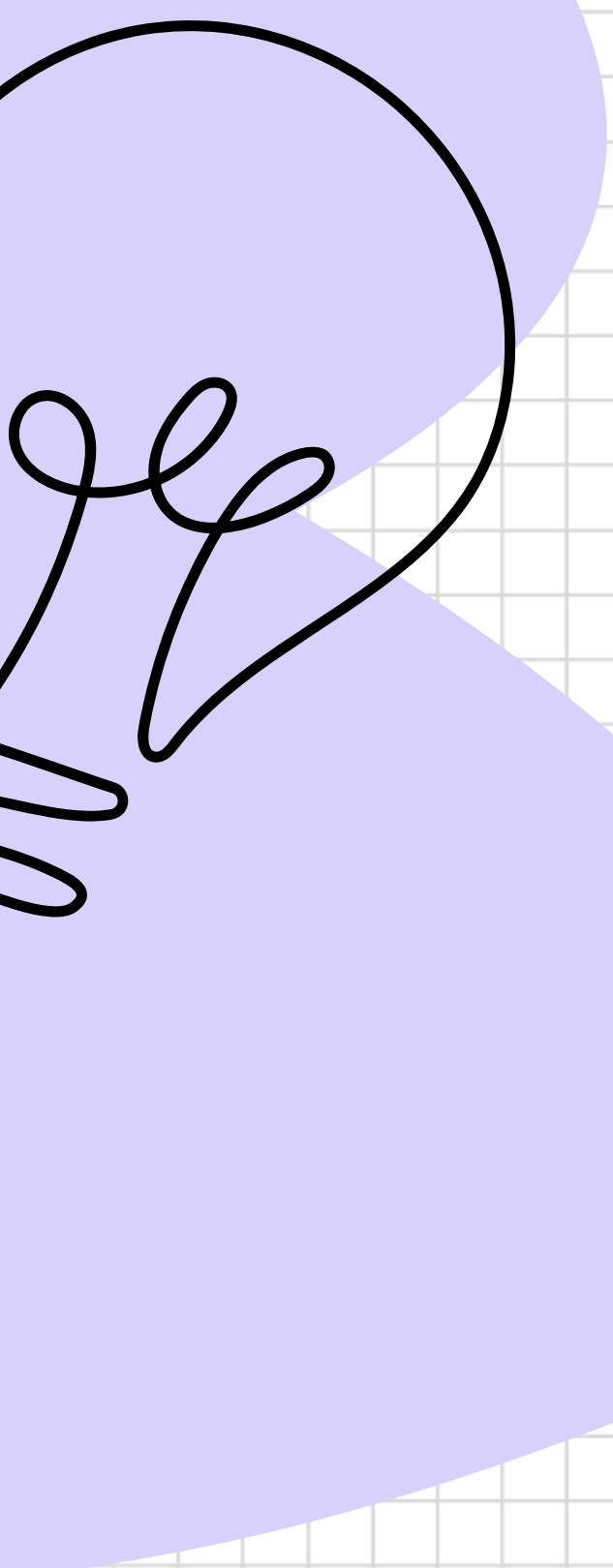




LARGE LANGUAGE MODELS

Marion Schaeffer
marion.schaeffer@ext.nexa.fr

Mai 2026



OBJECTIFS

01

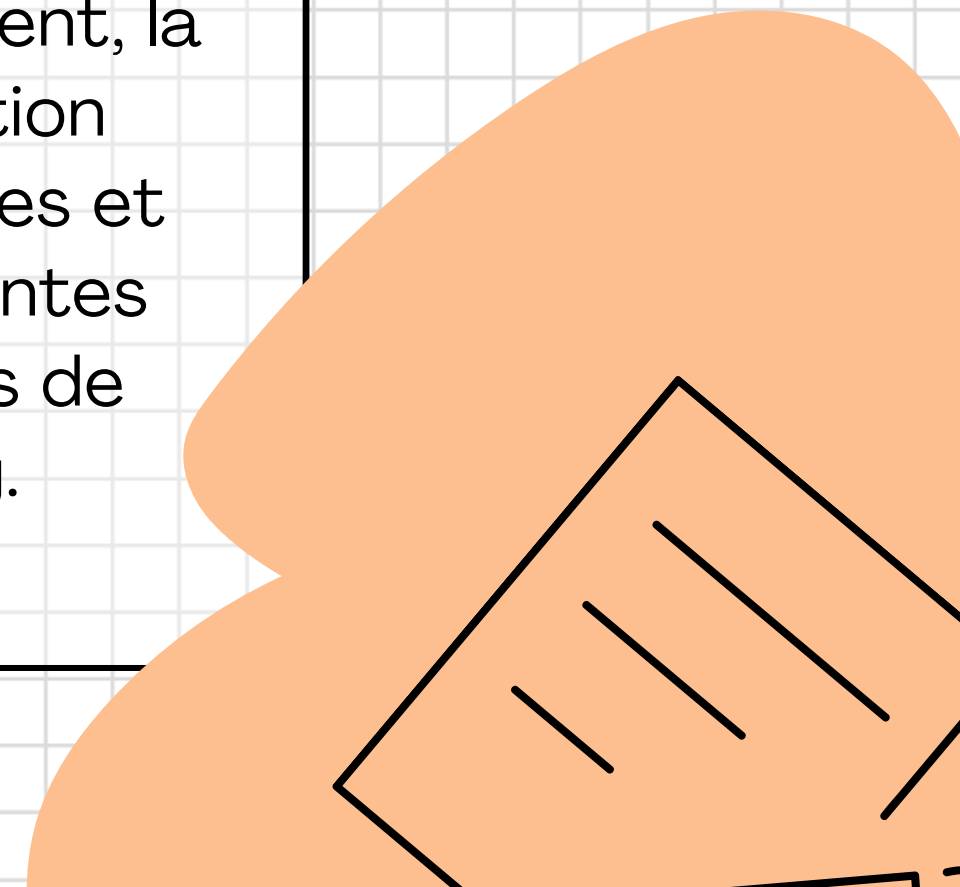
Savoir définir le traitement du langage naturel. Connaître l'historique et les représentation vectorielles.

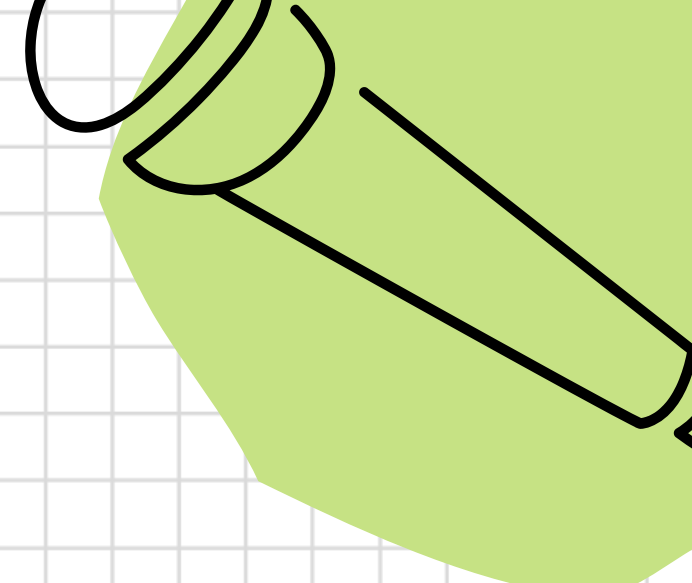
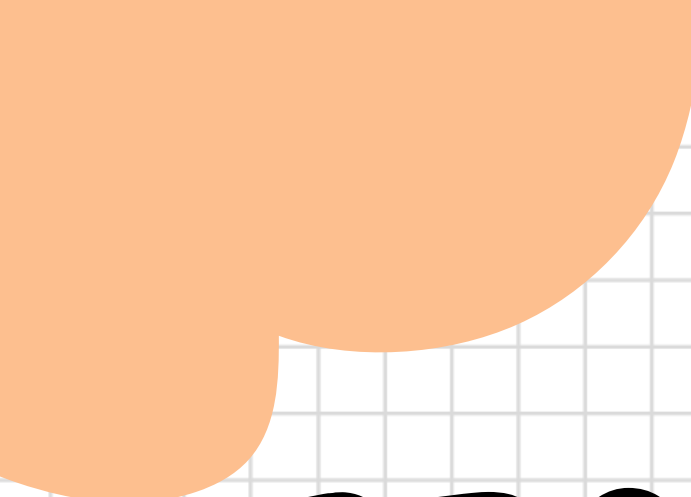
02

Maîtriser le mécanisme d'attention et l'architecture des Transformers avec les modèles de type Bert et GPT.

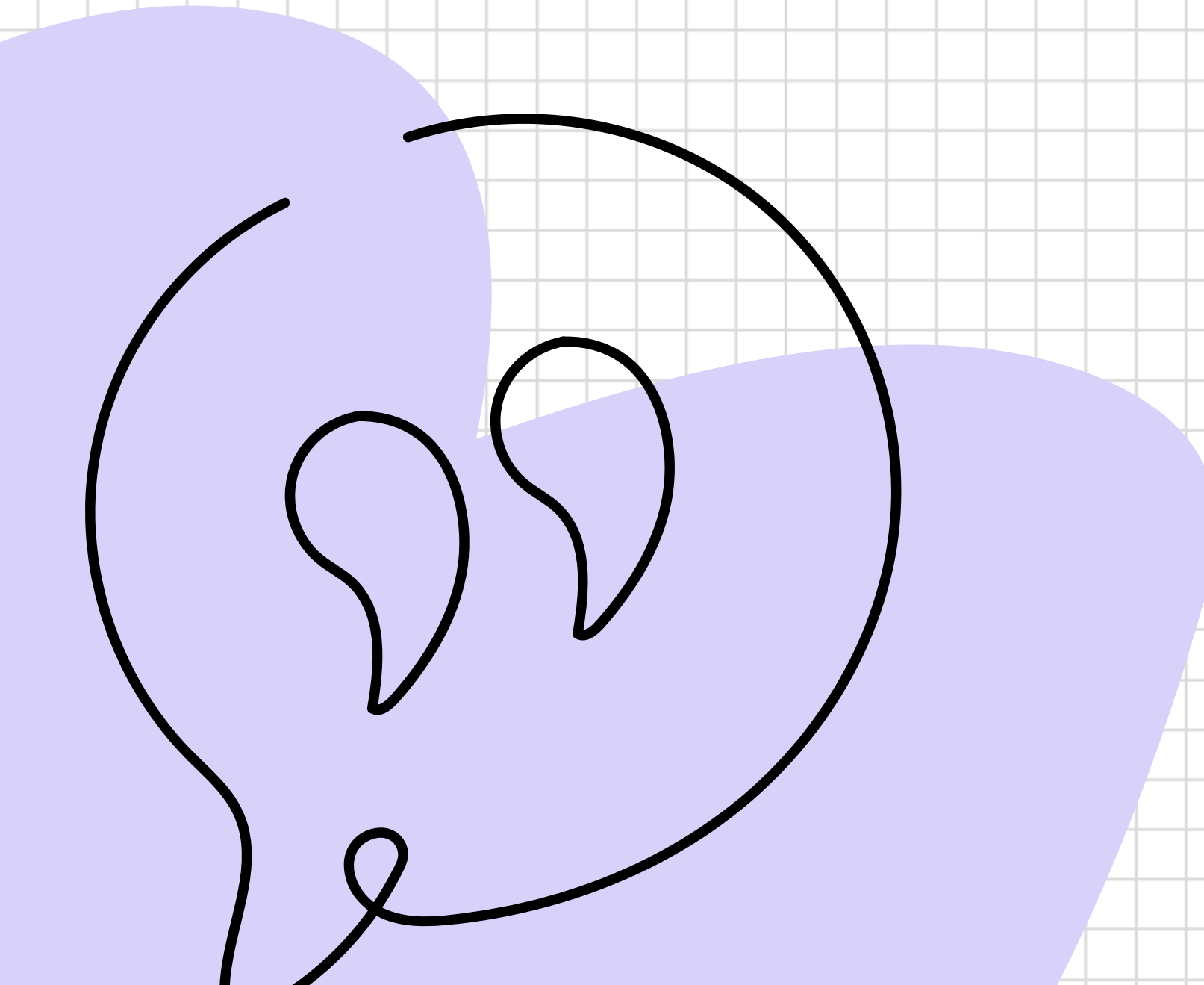
03

Comprendre le pré-entraînement, la spécialisation des modèles et les différentes techniques de prompting.





PROGRAMME



1.

Introduction au NLP

2.

Evolution des modèles

3.

Architecture des
Transformers

4.

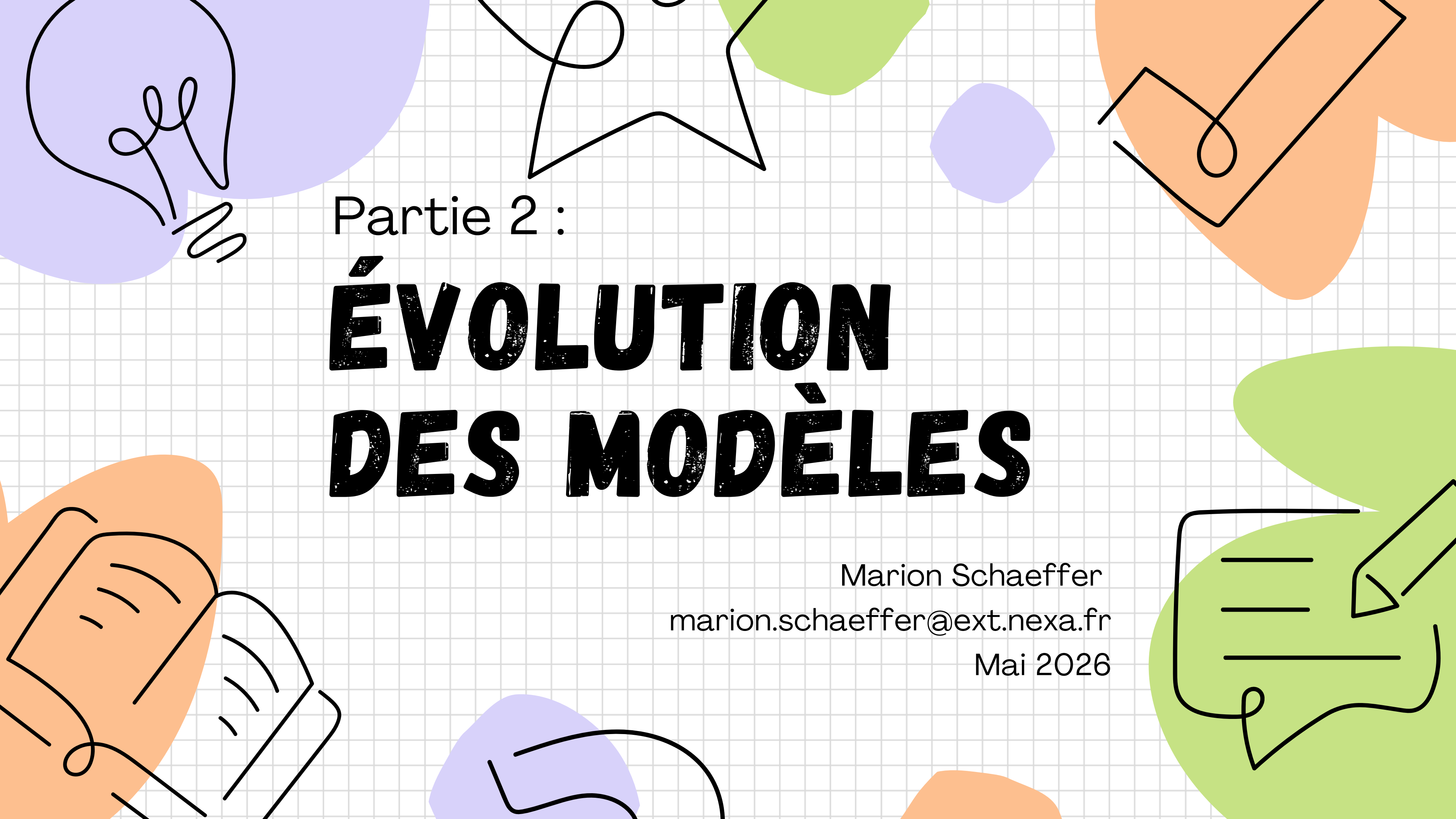
Bert

5.

GPT

6.

Fine-tuning et prompting



Partie 2 :

ÉVOLUTION DES MODÈLES

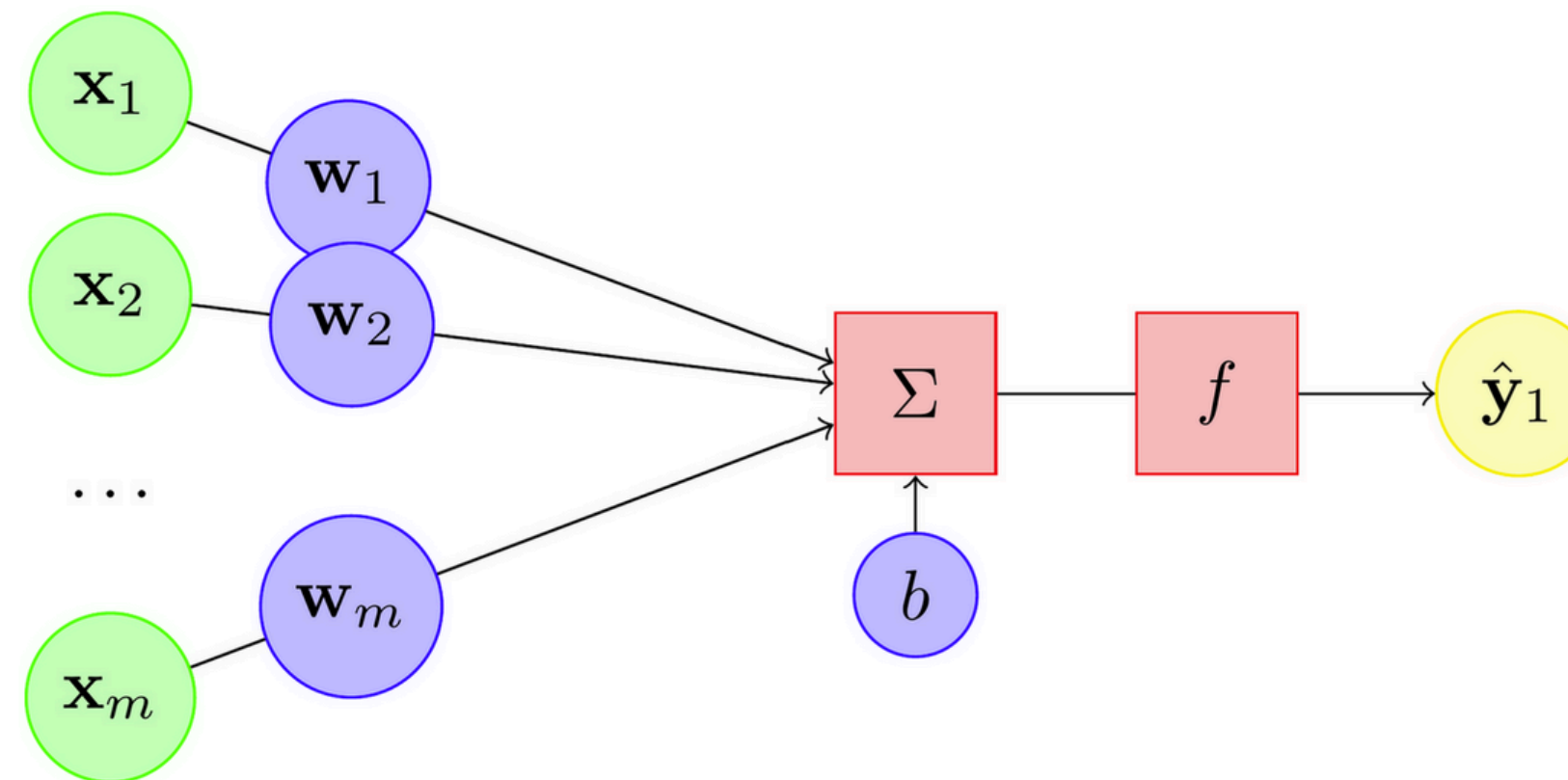
Marion Schaeffer

marion.schaeffer@ext.nexa.fr

Mai 2026

RÉSEAU DE NEURONES

Les réseaux de neurones artificiels sont des modèles mathématiques inspirés de la biologie. L'élément de base de ces réseaux, le neurone artificiel, est initialement issu de la volonté de modéliser le fonctionnement d'un neurone biologique.

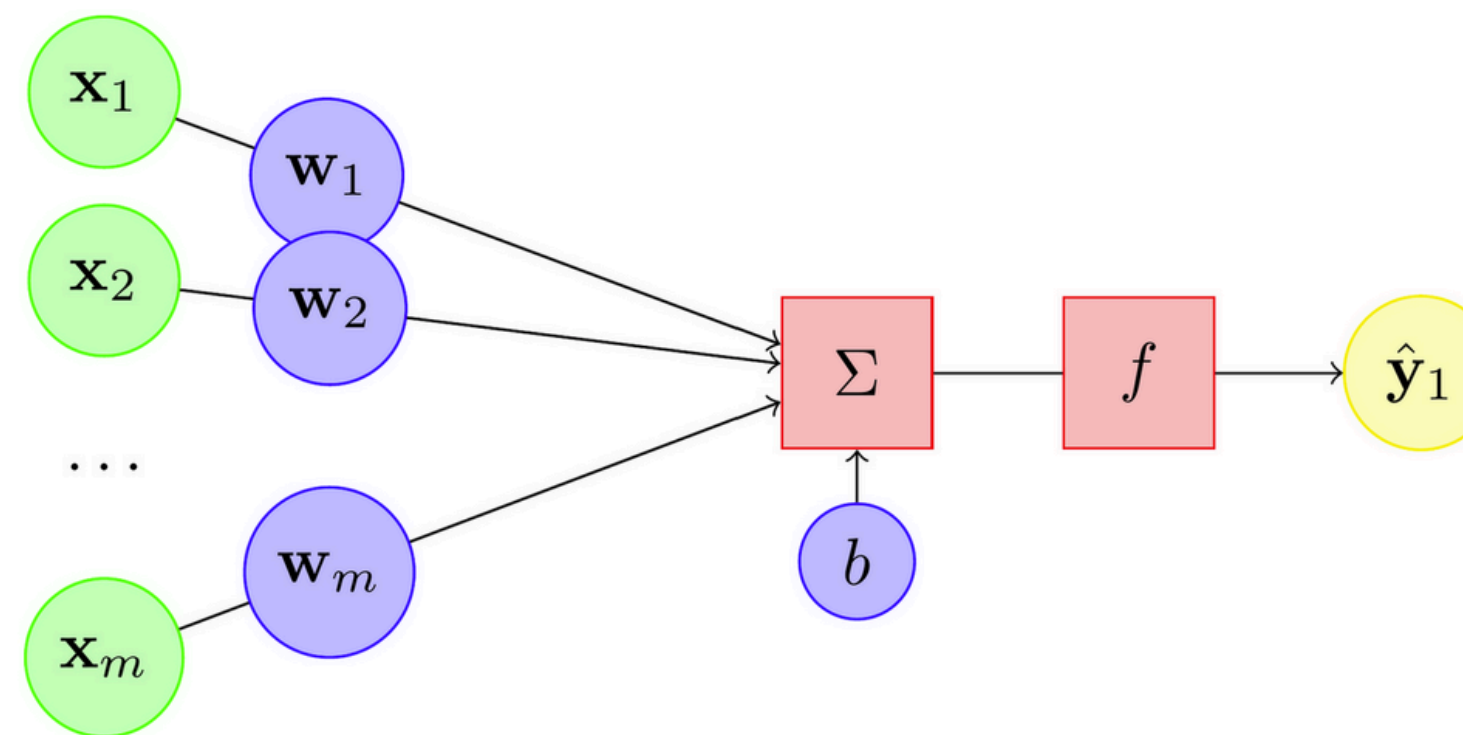


RÉSEAU DE NEURONES

Le perceptron, également appelé neurone artificiel ou neurone formel, cherche à reproduire le fonctionnement d'un neurone biologique.

Equation d'un neurone formel :

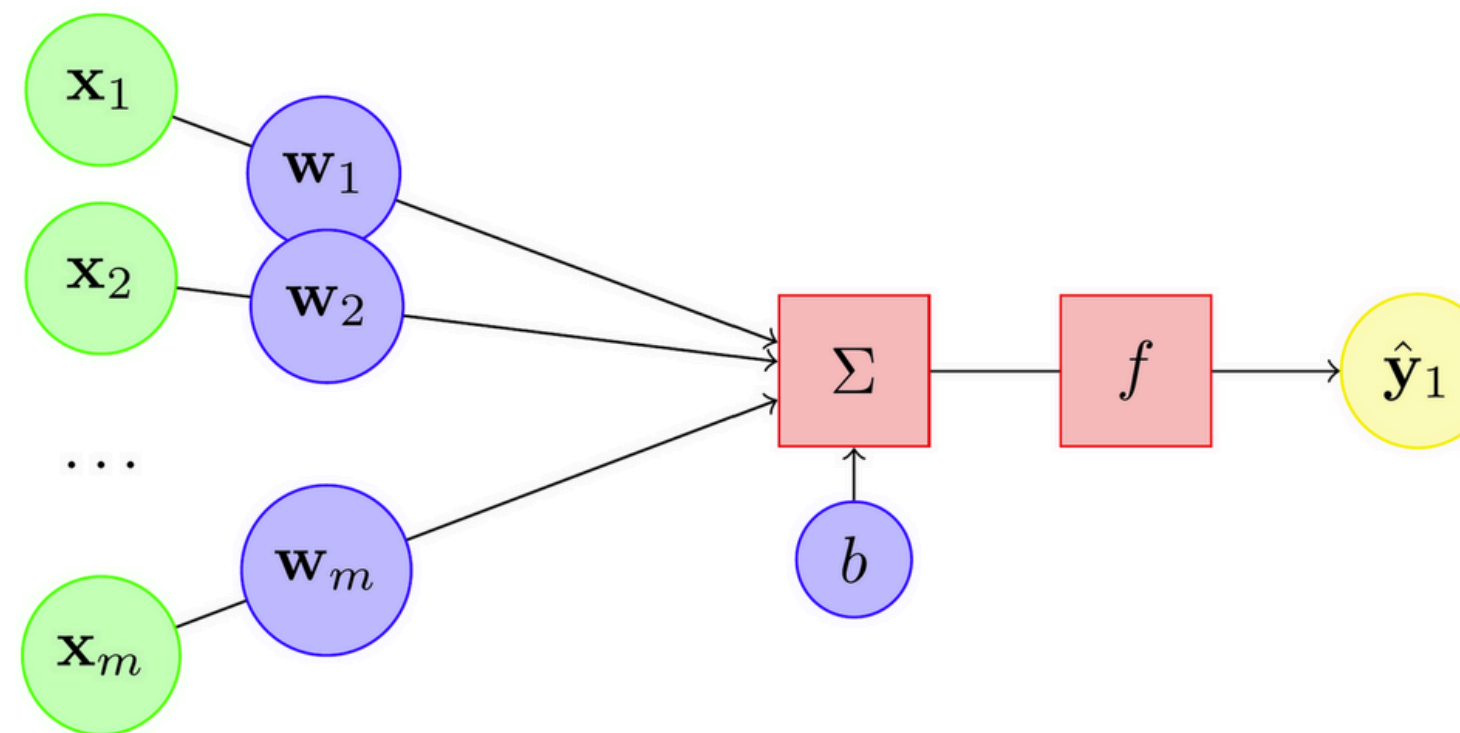
$$y = f (< w, x > + b)$$



RÉSEAU DE NEURONES

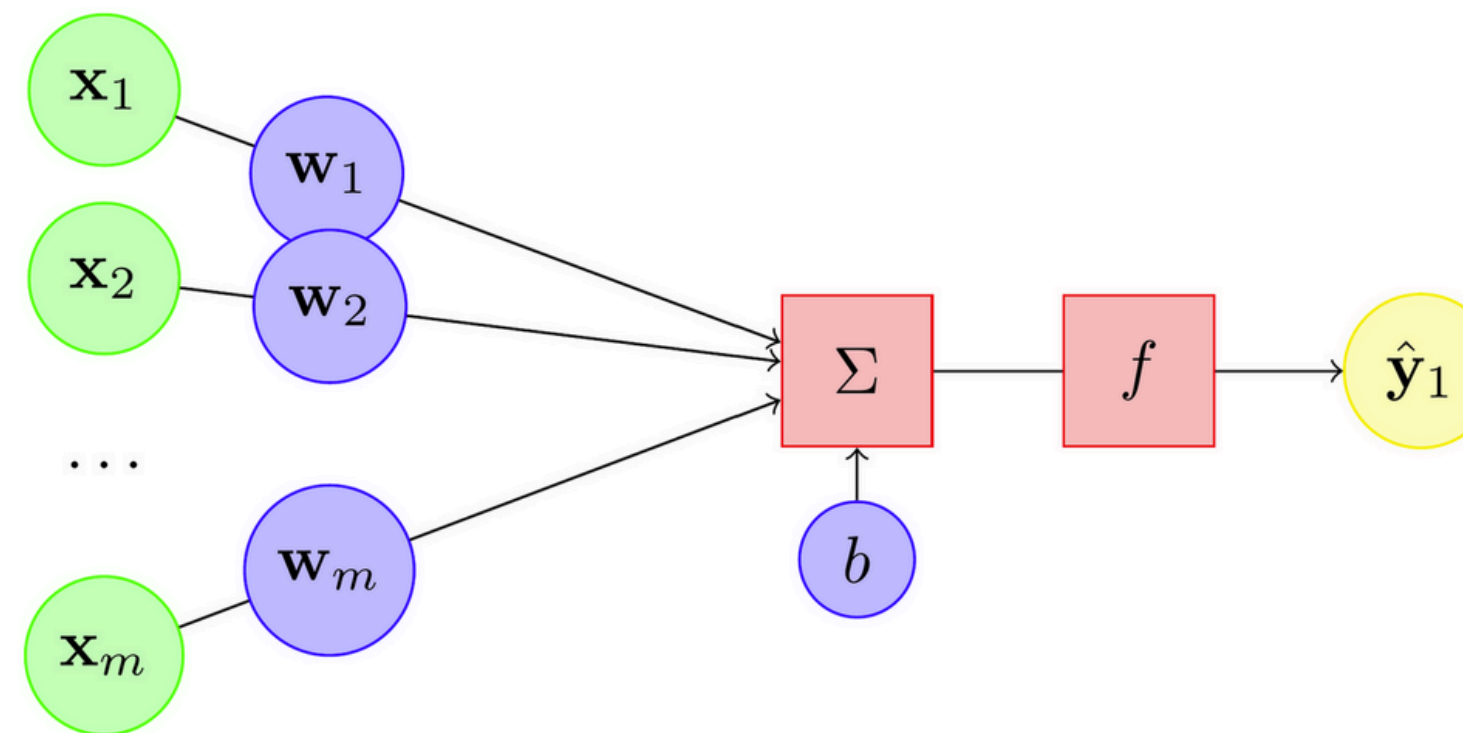
Les entrées sont variables. La construction du modèle fixe les paramètres. La sortie est calculée en fonction des entrées et des paramètres.

Chaque entrée est multipliée par un poids. Ensuite, toutes les entrées sont additionnées et ajoutées à un biais. Le résultat de la somme passe par une fonction de transfert (généralement non linéaire), qui produit alors la sortie souhaitée.



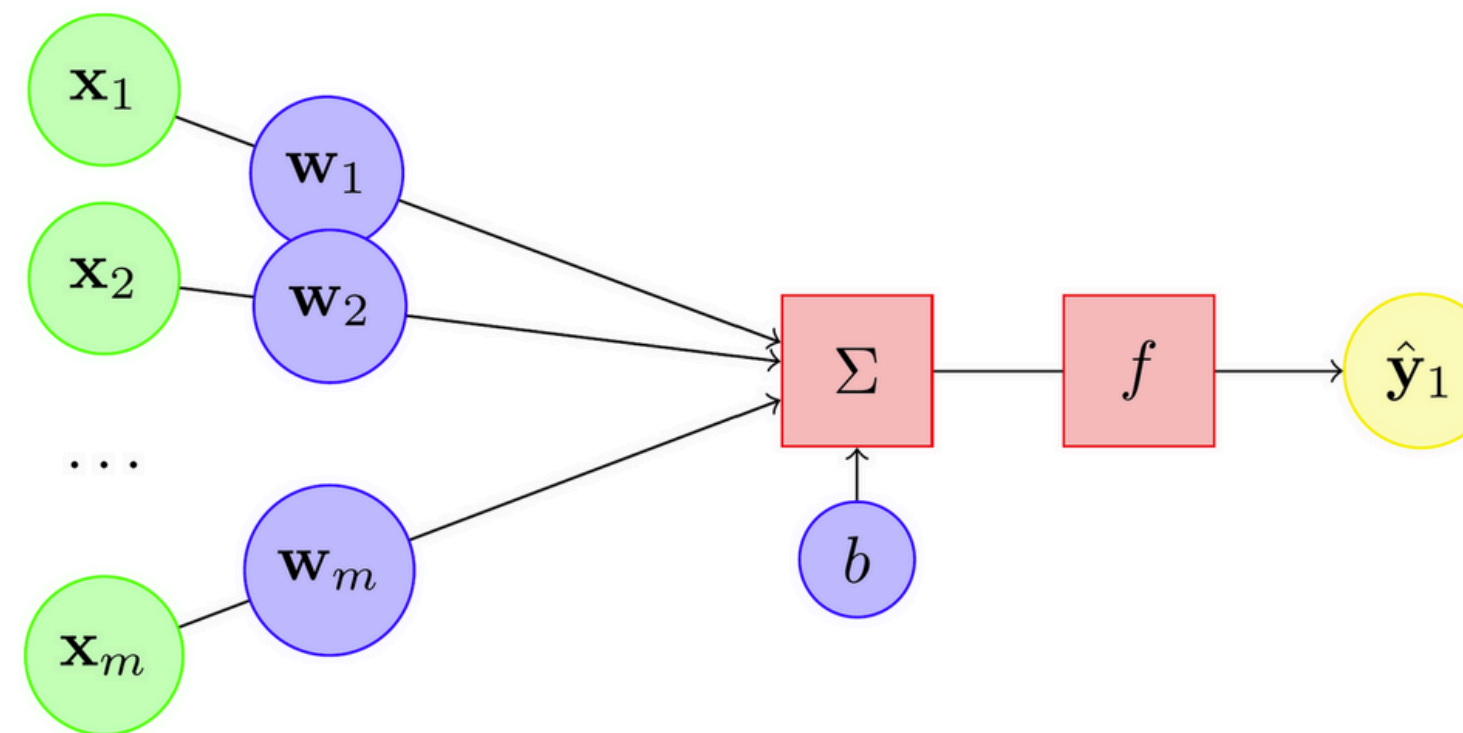
DESCENTE DE GRADIENT

Un neurone est entraîné en optimisant une fonction de perte. En d'autres termes, nous recherchons les paramètres w et b qui minimisent la fonction de perte. Les valeurs sont mises à jour à chaque itération jusqu'à ce que les valeurs de la fonction de perte deviennent stables. C'est l'algorithme de descente de gradient.



DESCENTE DE GRADIENT

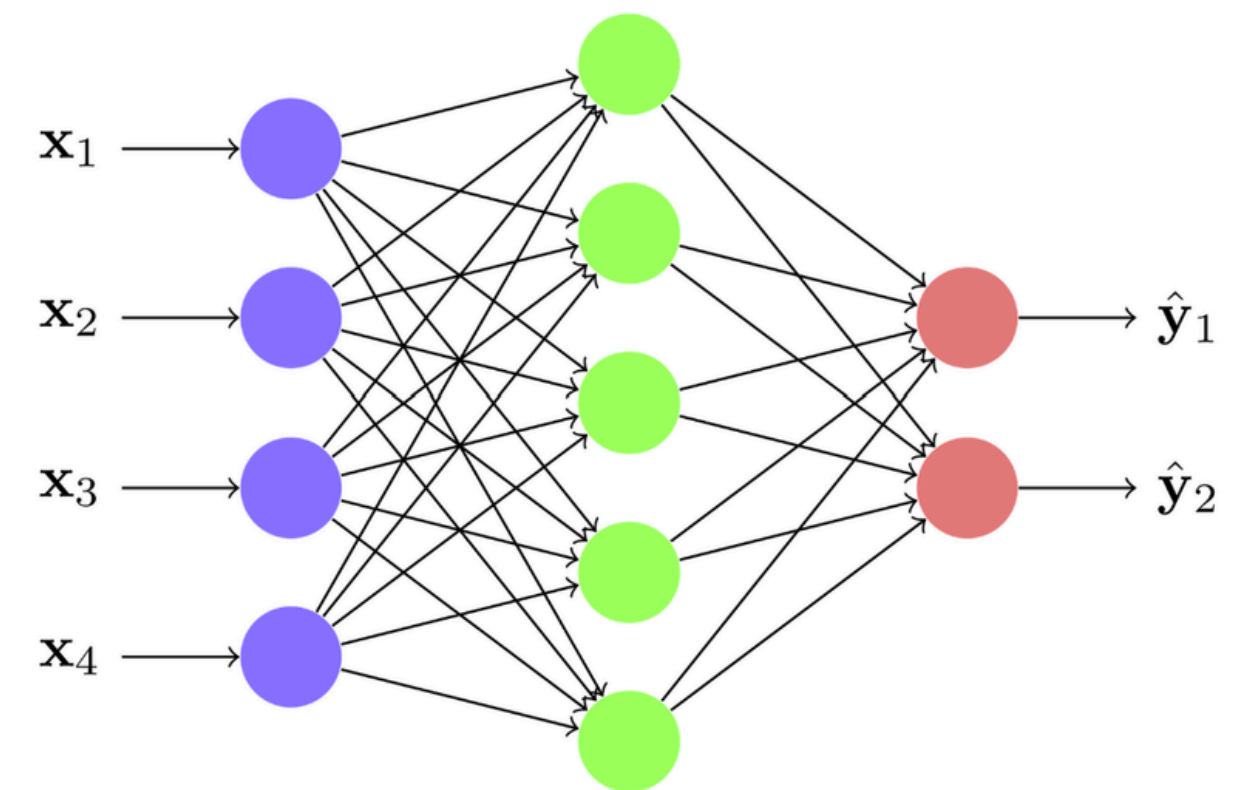
Un neurone est entraîné en optimisant une fonction de perte. En d'autres termes, nous recherchons les paramètres w et b qui minimisent la fonction de perte. Les valeurs sont mises à jour à chaque itération jusqu'à ce que les valeurs de la fonction de perte deviennent stables. C'est l'algorithme de descente de gradient.



RÉSEAUX MULTI COUCHES

Pour résoudre des problèmes complexes, les neurones biologiques communiquent également entre eux par l'intermédiaire de synapses, formant ainsi un réseau. Nous pouvons associer des neurones formels de la même manière en attribuant la sortie d'un neurone à une ou plusieurs entrées d'autres neurones.

Nous utilisons des réseaux organisés en couches. Le flux de données va toujours des couches d'entrée vers celles de sortie. Ils conviennent aux données de taille fixe. Ils sont appelés « Feed-Forward » ou « Multi-Layer Perceptron » (MLP).



RÉSEAUX MULTI COUCHES



RÉSEAUX SÉQUENTIELS

Les modèles standards (feedforward) ont une contrainte importante : la taille de l'entrée et la taille de la sortie doivent être identiques pour tous les exemples à traiter, tant dans l'apprentissage que dans la prise de décision.

Cela n'est pas viable pour le traitement de données séquentielles ou textuelles.

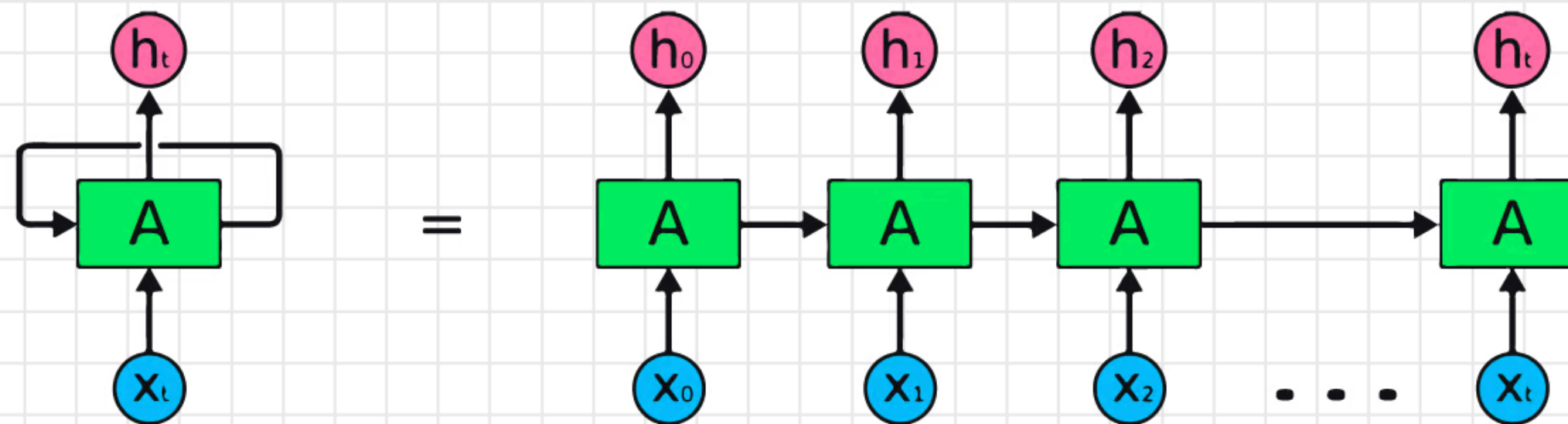
Une astuce très courante pour traiter des signaux de taille variable consiste à les analyser à l'aide d'une fenêtre glissante qui se déplace dans le sens de propagation du signal. Chaque fois que la fenêtre est déplacée, son contenu est soumis à un modèle statique.

Cependant, il existe généralement des dépendances entre les données d'entrée et/ou les signaux de sortie lors du traitement des signaux. On cherche alors à donner au classificateur la capacité d'avoir une mémoire. C'est le rôle des réseaux neuronaux récurrents (RNN).

RÉSEAUX DE NEURONES RÉCURRENTS

Les réseaux neuronaux récurrents (RNN) sont un type particulier d'architecture de réseau neuronal conçu pour traiter des données séquentielles. Les RNN tiennent compte de l'ordre des données d'entrée pour capturer les dépendances temporelles ou séquentielles.

L'entrée est x . L'état caché h est la mémoire interne du réseau, mise à jour à chaque instant t en fonction de l'entrée x et de l'état précédent. La sortie y est la prédiction ou le résultat à l'instant t .



RÉSEAUX DE NEURONES RÉCURRENTS

YouTube^{NZ}

Rechercher



Il est grand → RNN → He is tall

Les réseaux de neurones récurrents illustrés

A

Alexandre TL
9,29 k abonnés

S'abonner

👍 102



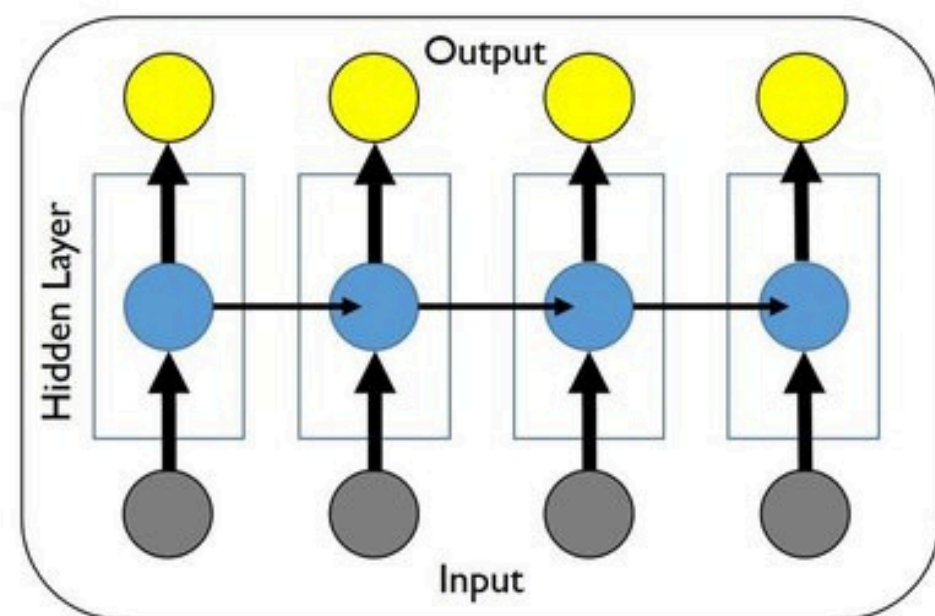
➦ Partager

✂️ Clip



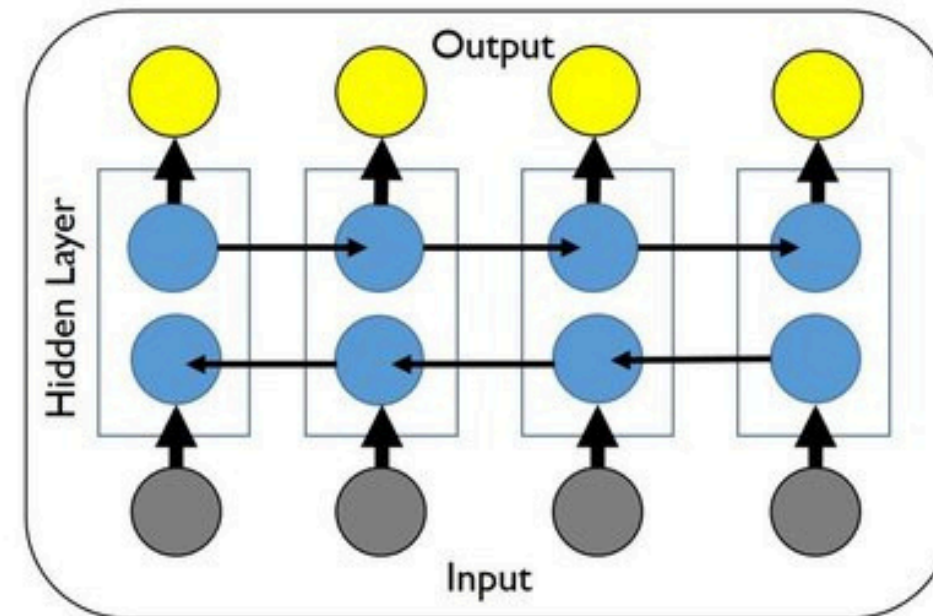
LSTM ET BiLSTM

Des architectures avancées de type RNN ont ensuite été proposées. Elles permettent d'intégrer des mécanismes de mémoire (LSTM) et de lire les séquences dans les deux sens (BiLSTM).



LSTM Architecture

Hochreiter & Schmidhuber, 1997



BiLSTM Architecture

Graves & Schmidhuber, 2005

LSTM

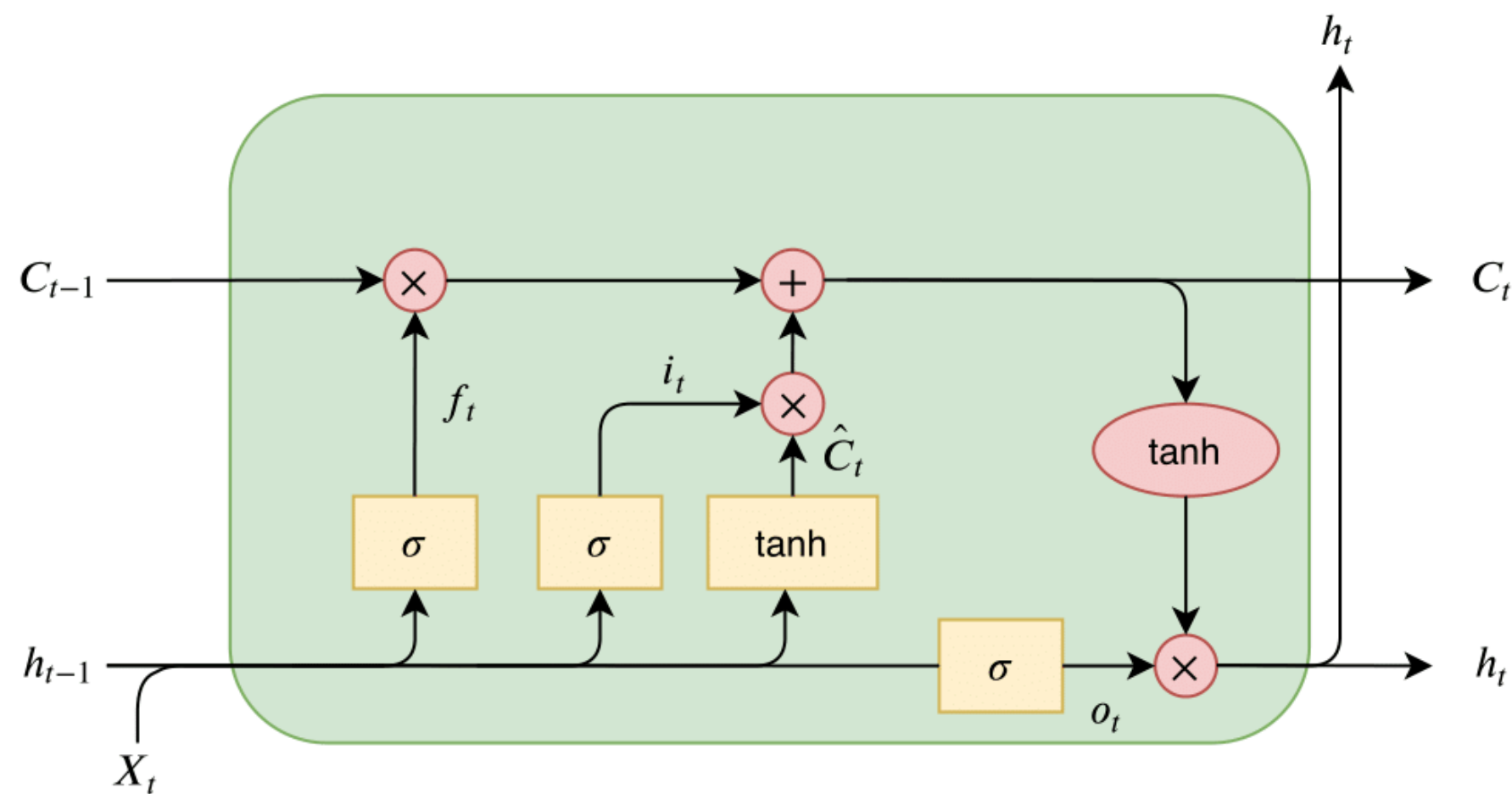
Les LSTM (Long Short-Term Memory) sont un type particulier de RNN conçus pour gérer les dépendances à long terme dans les données séquentielles. Ils utilisent des mécanismes spécifiques pour mémoriser ou oublier les informations pertinentes au fil du temps.

Les LSTM permettent de gérer des dépendances longues en capturant efficacement les relations temporelles dans de longues séquences.

Ils s'adaptent à des données séquentielles variées (texte, audio, séries temporelles).

Ils réduisent également les problèmes liés aux gradients qui disparaissent.

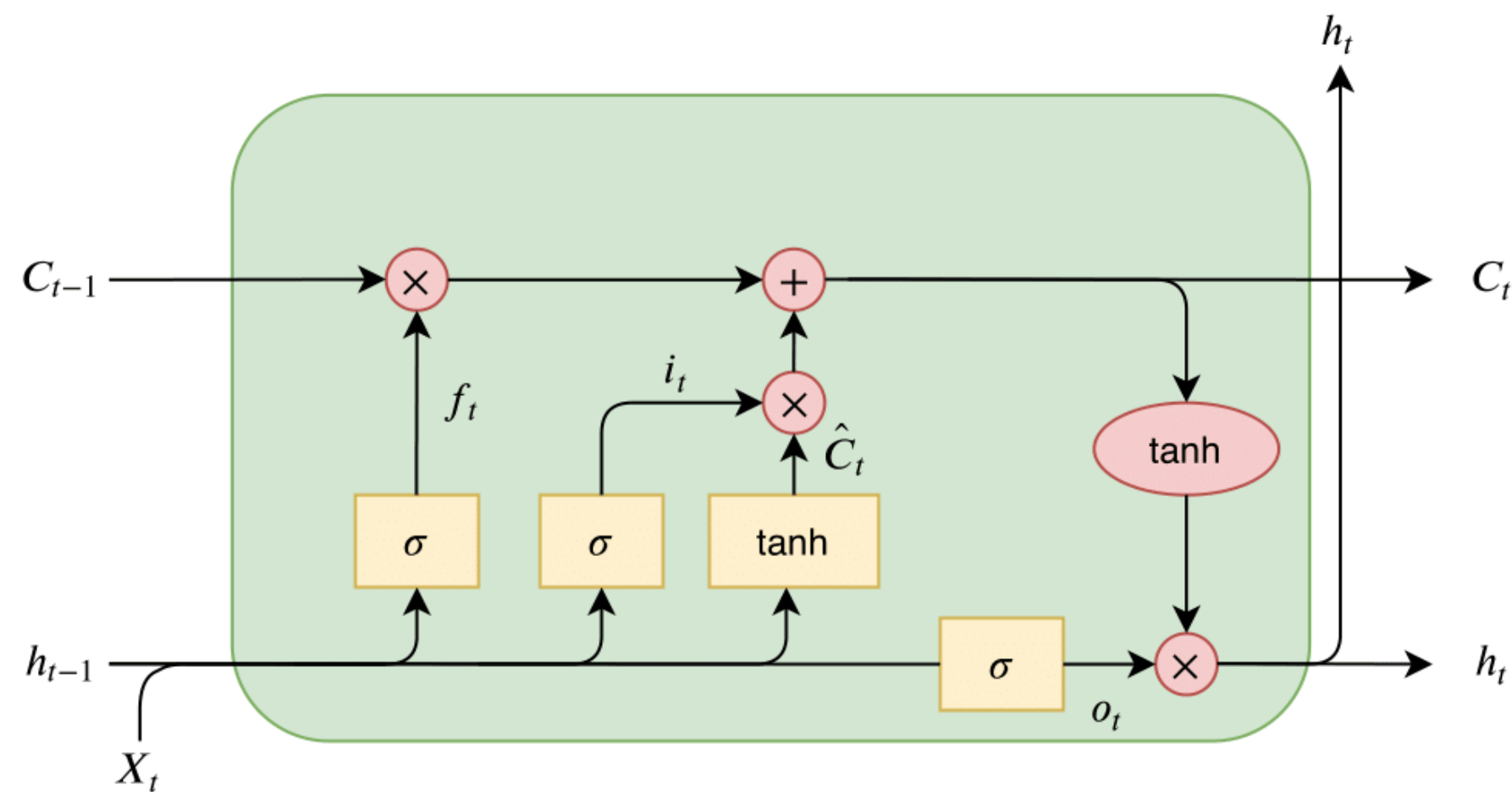
LSTM



Un LSTM se compose de cellules de mémoire avec trois mécanismes appelés portes :

- la porte d'oubli f_t filtre les informations de la cellule de mémoire précédente qui doivent être oubliées

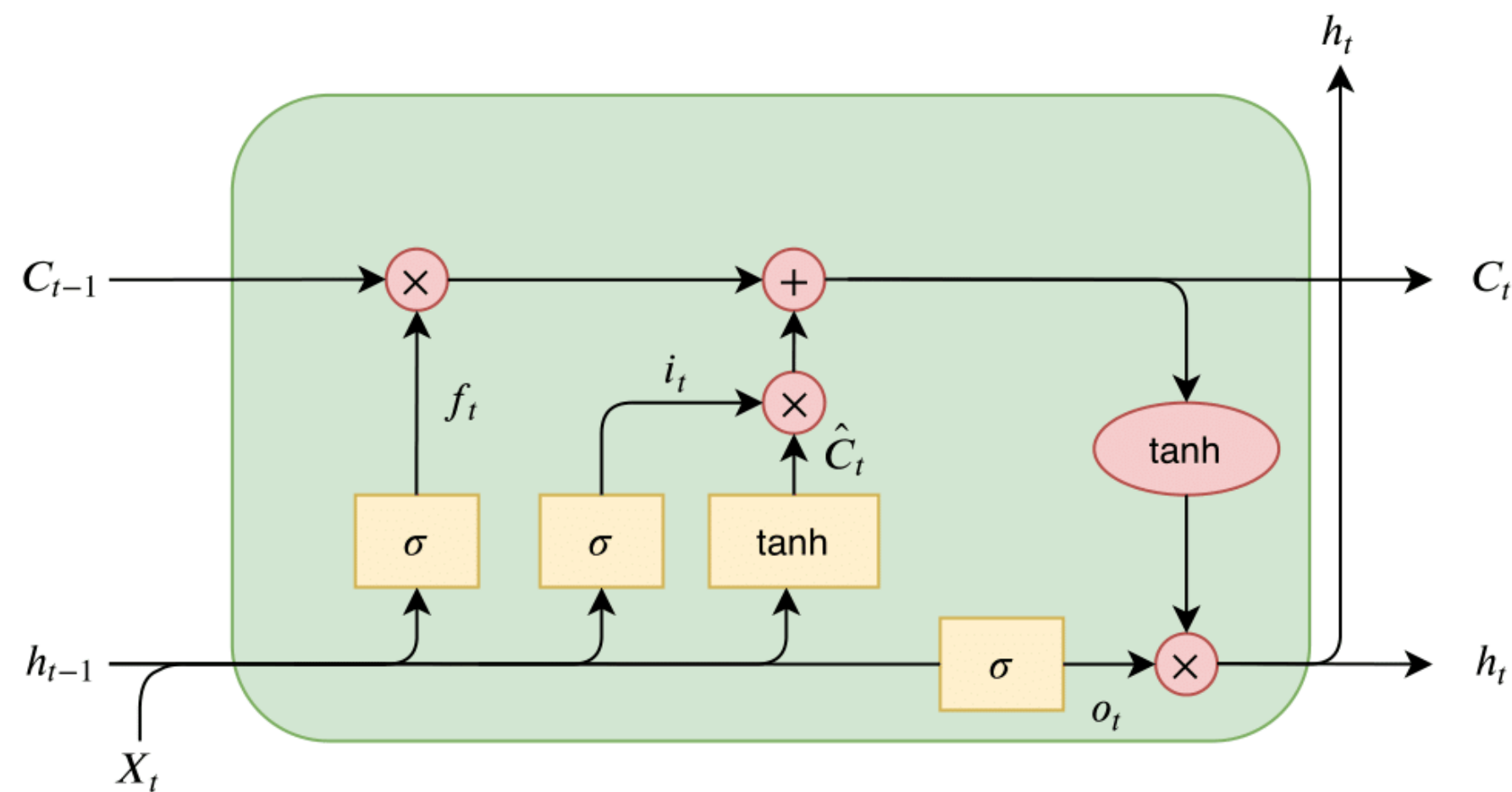
LSTM



Un LSTM se compose de cellules de mémoire avec trois mécanismes appelés portes :

- la porte d'entrée i_t détermine quelles nouvelles informations seront ajoutées à la cellule de mémoire, elle fonctionne avec un vecteur candidat C_t qui propose de nouvelles informations

LSTM



Un LSTM se compose de cellules de mémoire avec trois mécanismes appelés portes :

- la porte de sortie o_t décide quelle partie de la cellule de mémoire sera utilisée pour la sortie.

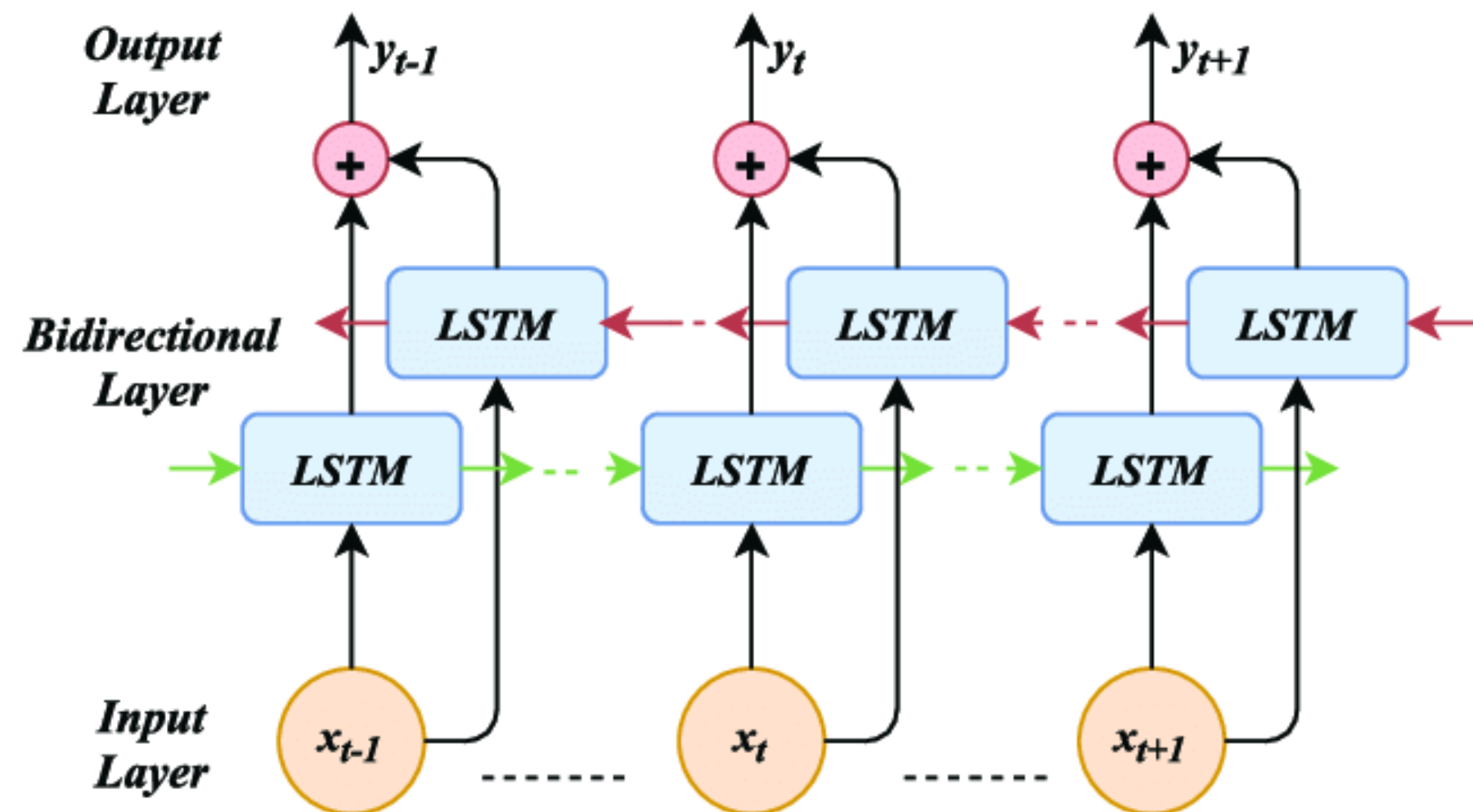
BiLSTM

Les BiLSTM (Long Short-Term Memory Bidirectionnels) sont une variante des réseaux LSTM, spécialement conçue pour mieux capturer les dépendances dans les données séquentielles en considérant non seulement le passé, mais aussi le futur. Cela est particulièrement utile dans des tâches où le contexte global d'une séquence est important.

Les BiLSTM capturent des dépendances avant et après chaque position, améliorant la compréhension du contexte. Cependant, le coût computationnel peut être important car les BiLSTM nécessitent deux passes sur la séquence, augmentant le temps d'entraînement et d'inférence.

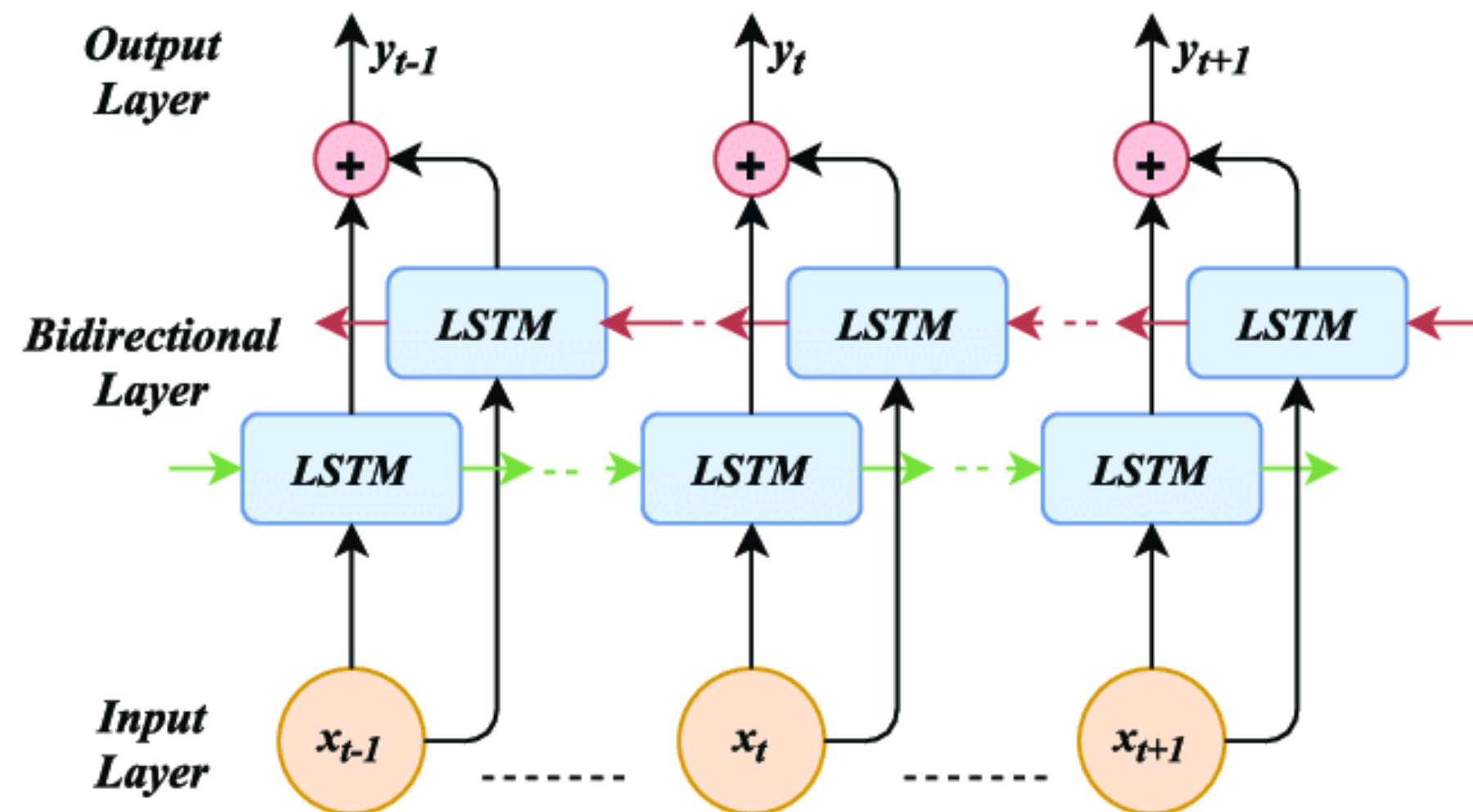
Comme pour les LSTM classiques, les très longues séquences peuvent poser problème, même si cela peut être partiellement résolu par l'utilisation de mécanismes d'attention.

BiLSTM



Un réseau BiLSTM est composé de deux LSTM distincts un LSTM avant (forward) qui lit la séquence dans le sens classique (de gauche à droite) et un LSTM arrière (backward) qui lit la séquence dans le sens inverse (de droite à gauche).

BILSTM



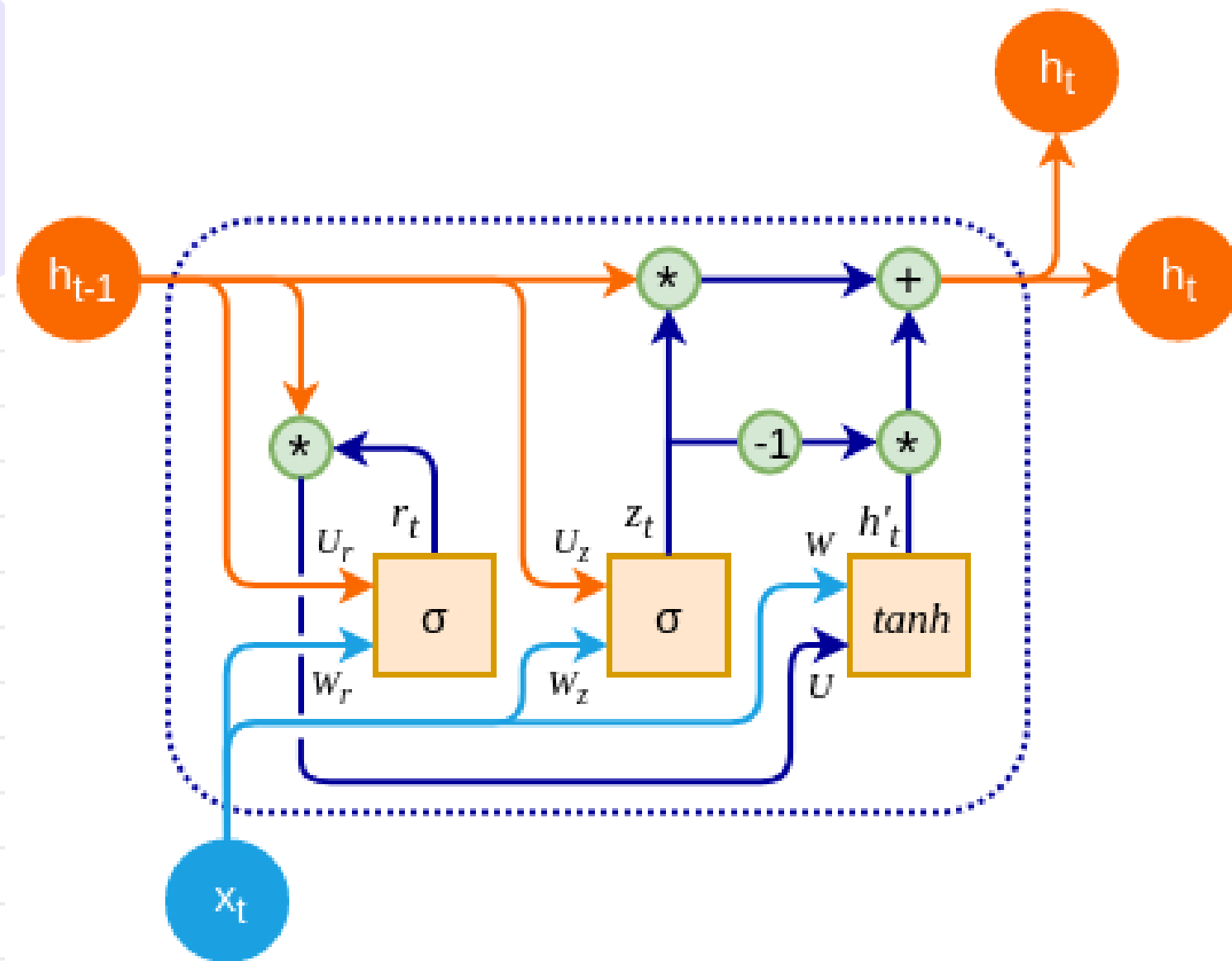
Les deux LSTM génèrent chacun un état caché pour chaque position dans la séquence. Les sorties des deux directions sont combinées, en les concaténant ou en prenant leur moyenne.

GRU

Les GRU (Gated Recurrent Units) sont une variante simplifiée des réseaux de neurones LSTM, conçus pour traiter les dépendances séquentielles dans les données tout en réduisant la complexité computationnelle. Ils utilisent un mécanisme de portes pour contrôler le flux d'information, permettant d'apprendre efficacement les dépendances à court et à long terme dans des séries temporelles ou d'autres données séquentielles.

Les GRU ont moins de paramètres que les LSTM, ce qui les rend plus rapides à entraîner et plus faciles à implémenter. Ils offrent des performances similaires aux LSTM pour de nombreuses tâches, tout en nécessitant moins de ressources computationnelles et conviennent aussi bien aux dépendances courtes que longues.

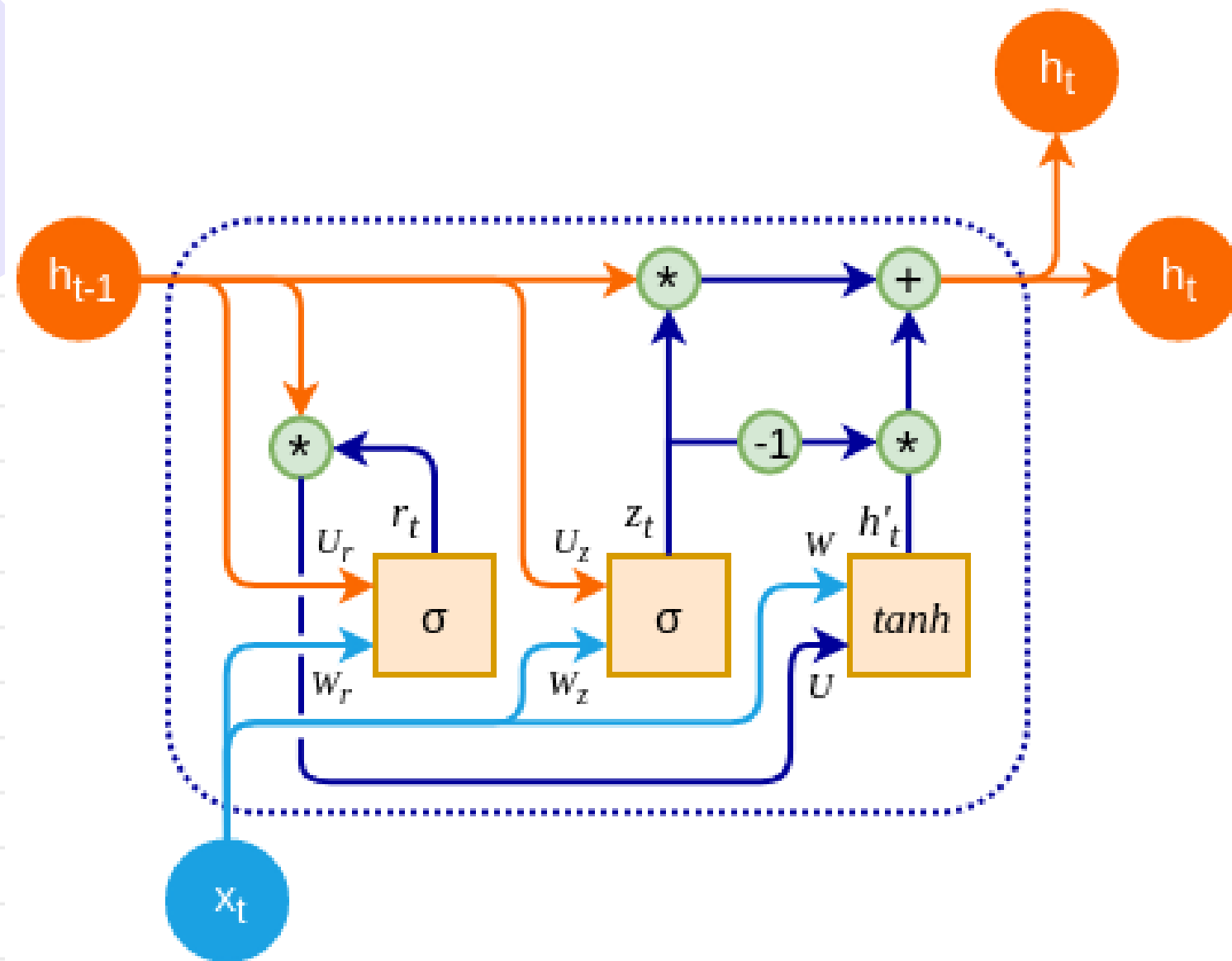
GRU



Les GRU remplacent la cellule de mémoire des LSTM par un mécanisme basé sur deux portes :

- la porte de réinitialisation r_t qui contrôle dans quelle mesure les informations de l'état précédent $h(t-1)$ doivent être oubliées ou réinitialisées avant de calculer l'état actuel.

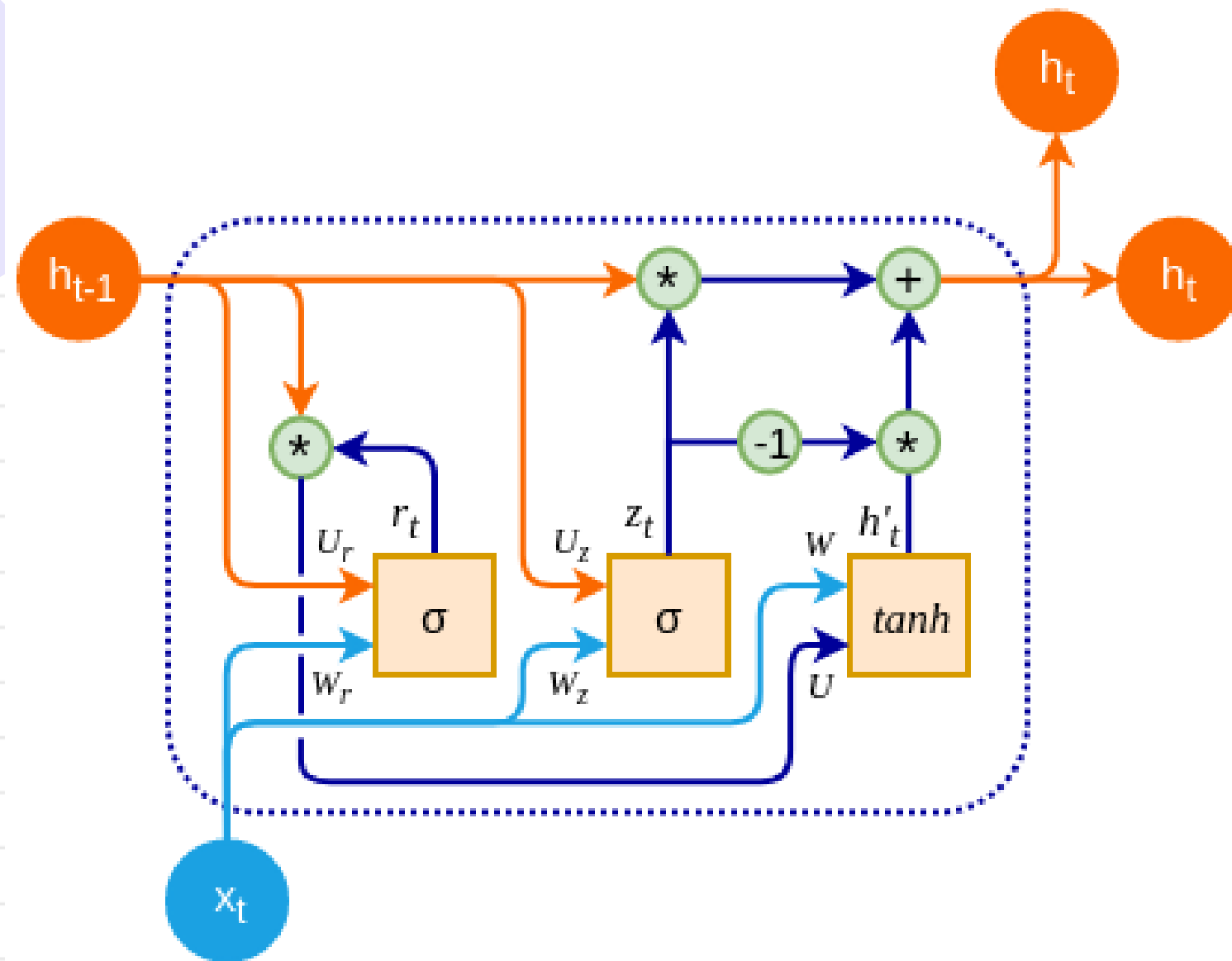
GRU



Les GRU remplacent la cellule de mémoire des LSTM par un mécanisme basé sur deux portes :

- la porte de mise à jour z_t détermine la quantité d'information issue de l'état précédent $h(t-1)$ qui sera conservée et la quantité d'information nouvelle $h'(t)$ qui sera ajoutée pour produire l'état actuel.

GRU



L'état caché actuel est une combinaison linéaire pondérée entre l'état précédent $h(t-1)$ et un nouvel état candidat $h'(t)$.